

Apunte 03 — Cuando internet dice que no: SSL, códigos de error y API keys

Proyecto Froth · aprender Machine Learning construyendo

Hoy chocamos con dos errores distintos seguidos. Vale oro saber distinguirlos, porque uno era *de tu máquina* y el otro *del servidor* — y se arreglan de formas opuestas.

1 Error 1: el candado (SSL / certificados)

Concepto: certificados y HTTPS

Cuando tu PC se conecta a `https://api.openalex.org`, antes de mandar nada verifica el **certificado** del servidor: su carnet de identidad digital, firmado por autoridades de confianza. Así sabes que hablas con OpenAlex de verdad y no con un impostor.

Python trae su propia lista de autoridades de confianza. En tu red (corporativa/con antivirus que inspecciona el tráfico), el certificado que llega no estaba en esa lista → Python se negó: `CERTIFICATE_VERIFY_FAILED`. Tu navegador sí funcionaba porque usa la lista de *Windows*, que sí conoce ese certificado.

Arreglo: la librería `truststore` le dice a Python “usa la lista de Windows”. La activamos una sola vez en `froth/__init__.py` y vale para todo el proyecto.

2 Error 2: la puerta cerrada (código 503)

Concepto: códigos de estado HTTP

Toda respuesta HTTP trae un número de 3 cifras que resume cómo fue:

- **2xx** — bien (200 OK).
- **4xx** — el error es *tuyo*: 404 no existe, 401 sin permiso, 429 pides demasiado rápido.
- **5xx** — el error es *del servidor*: 500 se rompió por dentro, 503 “no disponible” (saturado o en mantenimiento).

Nuestro 503 venía con un mensaje claro: “búsqueda anónima limitada por carga; usa una API key gratuita”. No había nada que arreglar en el código: era su puerta, cerrada para anónimos.

Ojo / error típico

Regla mental para siempre: **SSL/certificados** → **problema en tu lado** (red, Python, certificados). **5xx** → **problema en el lado de ellos** (esperar, identificarse o cambiar de servicio). Reintentar en bucle sin entender cuál de los dos es, es perder el tiempo.

3 La API key: tu carnet VIP (y por qué es un secreto)

Concepto: API key

Una **API key** es una cadena de texto que identifica tus peticiones ante el servicio. Con ella OpenAlex te da acceso estable (1 USD/día de uso gratuito, unas 1000 búsquedas — de sobra). Se envía como un parámetro más: `api_key=...`

Como te identifica *a ti*, es un **secreto**: quien la tenga gasta tu cuota en tu nombre. Por eso:

- **Nunca** se escribe dentro del código fuente.
- Vive en el archivo `.openalex_key` en la raíz del proyecto (una sola línea).
- Ese archivo está en `.gitignore`: la lista de cosas que Git (control de versiones) tiene prohibido subir a ningún sitio. Si un día publicas el código en GitHub, tu clave no viaja con él.

`config.py` la lee al arrancar (primero busca la variable de entorno `OPENALEX_API_KEY`; si no, el archivo) y `pull.py` la añade a cada petición.

Pruébalo tú (teclado en mano)

Comprueba tú mismo que la clave está donde debe y fuera del alcance de Git:

1. Abre `.gitignore` (con el Bloc de notas) y localiza la línea `.openalex_key`.
2. En PowerShell, dentro de la carpeta del proyecto: `Get-Content .openalex_key` (verás tu clave — solo tú).

En una frase

SSL falló en tu máquina (arreglado con `truststore`); el 503 era el servidor saturado rechazando anónimos (arreglado con API key); y una API key es un secreto que vive fuera del código y fuera de Git.